

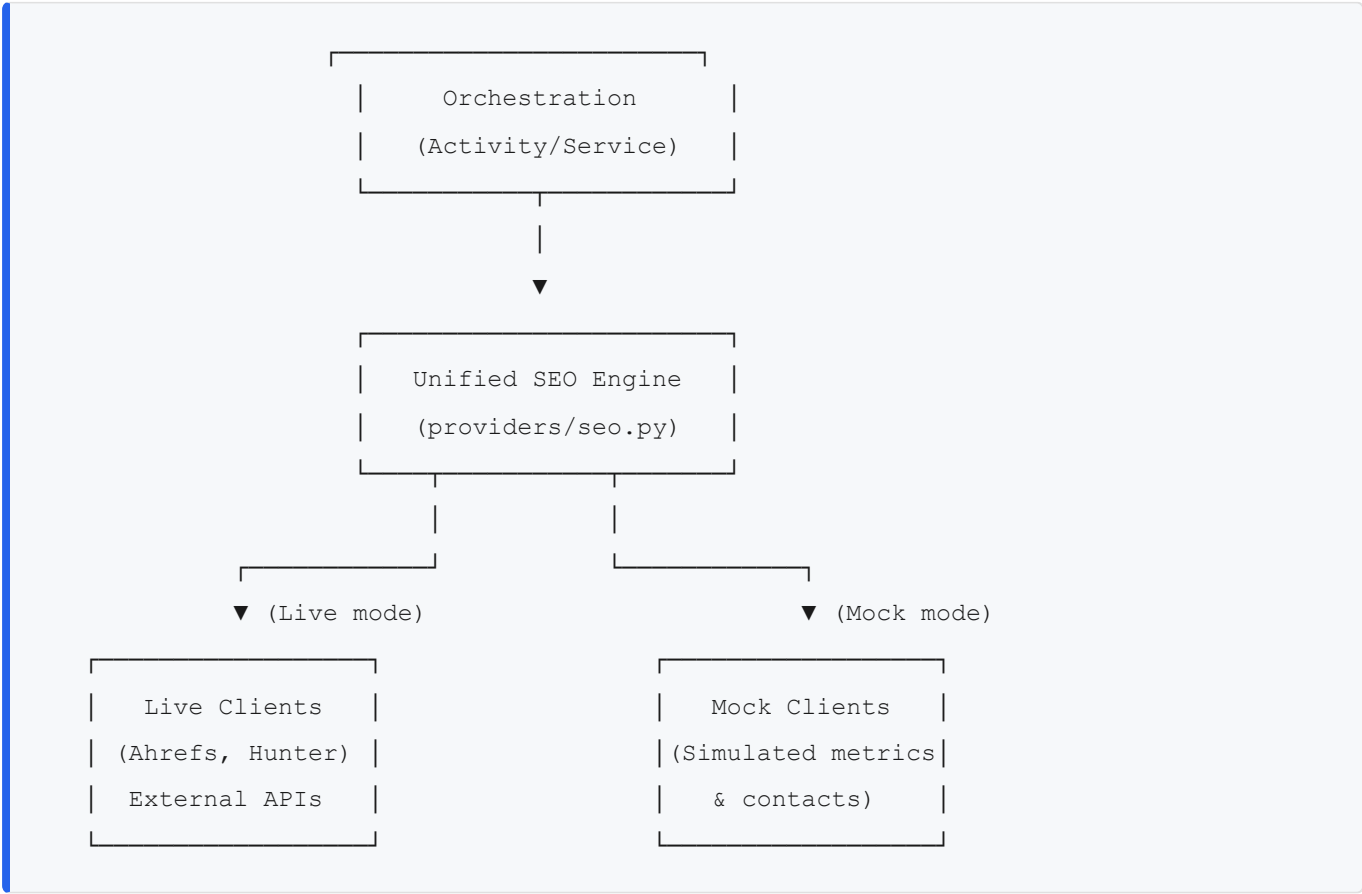
PROJECT 31A — EXTERNAL PROVIDER INTEGRATION BIBLE (DOCUMENT 9)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

1. UNIFIED PROVIDER LAYERING

Project 31A is designed to be provider-agnostic. While it integrates directly with high-tier search engine metrics databases (Ahrefs), contact discovery registries (Hunter.io), and search volume aggregators (DataForSEO), it routes all requests through a unified provider interface layer declared in `backend/src/seo_platform/providers/seo.py`.



This abstraction ensures that the platform can switch between live service integrations and zero-cost simulated mocks seamlessly, depending on environment variables and tenant configurations.

2. METRICS & COMPETITOR INTERSECT (AHREFS CLIENT)

The primary SEO metric database is **Ahrefs**. The client is implemented in `backend/src/seo_platform/clients/ahrefs.py`.

2.1 Main Operations

1. `get_referring_domains(domain: str, limit: int = 50)` :
 - Calls Ahrefs v3 endpoint `/v3/site-explorer/referring-domains`.
 - Parses response into target opportunities: `{domain, domain_rating, backlinks_count}`.
 - Restricts results to domains with DR ≥ 20 by default (unless campaign overrides apply).
2. `get_domain_metrics(domain: str)` :
 - Queries `/v3/site-explorer/domain-rating` to fetch the Domain Rating (DR) and URL Rating (UR).

2.2 Error Handling and Rate Limit Handling

- **AhrefsRateLimitError**: Triggers when Ahrefs returns HTTP 429. The client catches this and immediately raises `AhrefsRateLimitError` to the executing workflow activity.
 - **Failover**: If Ahrefs fails, the activity falls back to the configured fallback engine in `providers/seo.py`, which maps to DataForSEO or OpenPageRank to query metrics.
-

3. CONTACT DISCOVERY (HUNTER.IO CLIENT)

Email scraping and mailbox verification are routed through the **Hunter.io** client in `backend/src/seo_platform/clients/hunter.py`.

3.1 Contact Searching Pipeline

1. `domain_search(domain: str, limit: int = 3)` :
 - Queries `/v2/domain-search?domain={domain}&api_key={key}`.
 - Extracts contact names, emails, and confidence scores.
 - Filters out generic department emails (e.g. `info@`, `support@`) in favor of personal emails (`firstname.lastname@`).
 2. `verify_email(email: str)` :
 - Queries `/v2/email-verifier?email={email}&api_key={key}`.
 - Parses the verification status:
 - `deliverable` : Safe to send.
 - `risky` : Acceptable for manual review but blocked for autonomous dispatch.
 - `undeliverable` : Rejected immediately.
-

4. KEYWORD & SERP AGGREGATOR (DATAFORSEO CLIENT)

Keyword search volume lookup, search intent classification, and SERP listings are fetched via DataForSEO in `backend/src/seo_platform/clients/dataforseo.py`.

4.1 Key Capabilities

- `expand_seed_keywords(seeds: List[str], lang: str = "en")`:
 - Submits POST request to `/v3/seo_and_web_data/google/keyword_ideas/live`.
 - Extracts related keyword terms, search volumes, and competition levels.
 - `get_serp_listings(keyword: str, geo: str = "us")`:
 - Queries `/v3/serp/google/organic/live` to retrieve the top 100 ranking URLs.
 - Used by the competitor discovery and SERP volatility snapshot agents.
-

5. WEB SCRAPING & CRAWLING STACK

To personalize emails and verify backlink placements, the system implements a multi-tier crawling framework under `backend/src/seo_platform/clients/`.

5.1 Scrapling (`scrapling.py`)

- **Purpose:** Primary HTTP scraper designed for speed.
- **Features:** Uses customized HTTP headers, rotates user agents, and maintains cookie sessions to bypass standard Cloudflare/WAF checks on target domains. Includes a Redis-backed caching layer (`scrapling_cache.py`) with a 24-hour TTL to prevent redundant requests.

5.2 Firecrawl (`firecrawl.py`)

- **Purpose:** Browser-rendering scraper (fallback).
- **Features:** Used when target websites rely heavily on React/Vue client-side rendering. Simulates full page load, executes JS, and extracts text content.

5.3 Trafilatura (`trafilatura.py`)

- **Purpose:** Content extraction utility.
- **Features:** Parses raw HTML returned by Scrapling or Firecrawl. Discards navigation bars, sidebars, and footer boilerplate text. Extracts clean body text for LLM contextual grounding.

5.4 Wappalyzer (`wappalyzer.py`)

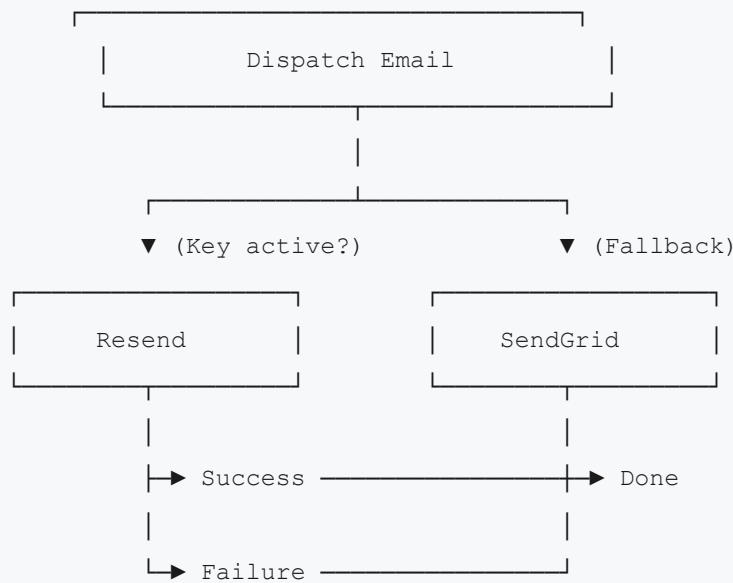
- **Purpose:** Technology stack fingerprinting.

- **Features:** Parses scripts and response headers to identify if the target prospect runs on WordPress, Shopify, Webflow, etc., allowing the outreach generation activity to personalize the email.

6. EMAIL DISPATCH & DELIVERY RELIABILITY

Outgoing outreach emails are managed by the unified mail delivery system in `backend/src/seo_platform/services/email/email_provider.py`.

6.1 Multi-Provider Routing Sequence



1. **Primary Provider (Resend):** Used for outreach campaigns because of high delivery rates and simple API key scoping.
2. **First Fallback (SendGrid):** If Resend returns an authentication error, rate limit, or network failure, the client falls back to SendGrid.
3. **Second Fallback (Mailgun):** Used if both Resend and SendGrid fail.
4. **Development Sandbox (MailHog):** In non-production environments, all SMTP traffic is redirected to MailHog (localhost:1025) to prevent accidental spamming of live contacts during testing.

7. MOCK VS. LIVE SWITCHING LOGIC

- **Trigger:** Configured via `USE MOCK PROVIDERS` inside `.env`.
- **Implementation:**

`providers/seo.py` evaluates this variable on boot. If set to `true` (valid in development and testing only), it mounts classes from `backend/src/seo_platform/providers/mock/` instead of the live clients.

- **Mock Actions:**

- Mock Ahrefs generates random referring domains with a deterministic seed based on campaign inputs.
- Mock Hunter returns `deliverable` for any email address ending in `@gmail.com` or `@example.com` and `undeliverable` for others.
- Mock Email Provider logs the body text of sent emails to local JSON files in the `/scratch/` directory.

8. CREDENTIAL VAULT INTEGRATION

External API keys are never stored in plain text configuration files. They are fetched on-demand from the credential vault.

- When an activity needs to call Ahrefs, it executes `provider_key_service.get_key(tenant_id, "ahrefs")`.
- The service retrieves the encrypted blob from the `ProviderKey` table.
- Decrypts the key using the verified `ENCRYPTION_MASTER_KEY` context.
- Passes the key to the short-lived client instance.

9. PROVIDER HEALTH CENTER

The platform monitors provider availability in `backend/src/seo_platform/services/provider_health.py`.

- **Telemetry:** Logs response times and error rates (HTTP 4xx, 5xx) for each provider endpoint.
- **Circuit Breaker:** If a provider (e.g. Hunter.io) fails 5 times consecutively, the health center marks the provider as `unhealthy` and trips the circuit breaker.
- **Orchestration Change:** Downstream activities check provider status. If the primary provider's circuit is tripped, they automatically fall back to alternative providers or mock simulations to prevent campaign execution blocks.